

Week 3: Seller and Inventory Management System

Scenario:

The e-commerce company sells products supplied by multiple sellers. The system should maintain seller details, track products supplied by each seller, and monitor inventory levels to ensure product availability.

1. Create Seller and Inventory Tables

2. Establish Relationships

Seller and Product

One seller can supply multiple products.

One product can be supplied by one or multiple sellers.

Relationship:

Seller → Product is one to many

Product and Inventory

Each product has inventory information.

Inventory tracks available stock.

Relationship:

Product → Inventory is one to one

```
SQL> CREATE TABLE Seller(Seller_ID VARCHAR(7),Seller_Name VARCHAR(25),Email
VARCHAR(30),Phone NUMBER(10),Address VARCHAR(45));

Table created.

SQL> CREATE TABLE Inventory(Inventory_ID VARCHAR(7),Product_ID VARCHAR(7),Se
ller_ID VARCHAR(7),Stock_Quantity NUMBER,Stock_Status VARCHAR(11),Last_Updat
ed DATE);

Table created.
```

Shows the CREATE TABLE commands used to create the Seller and Inventory tables.

Primary Key added for Both Tables:

```
SQL> ALTER TABLE CATEGORY ADD CONSTRAINT PK_CATEGORY PRIMARY KEY (CATEGORY_ID);  
Table altered.  
  
SQL> ALTER TABLE PRODUCT ADD CONSTRAINT PK_PRODUCT PRIMARY KEY (PRODUCT_ID);  
Table altered.
```

Shows the primary key constraints added to the Category and Product tables.

Foreign Key for Inventory Table:

```
SQL> ALTER TABLE Product ADD CONSTRAINT FK_Product_Category FOREIGN KEY (Category_ID) REFERENCES Category(Category_id);  
Table altered.  
  
SQL> ALTER TABLE Category MODIFY Category_Name VARCHAR(100) NOT NULL;  
Table altered.
```

Shows the foreign key constraint added to the Inventory table.

Constraints to make Unique Email ID:

```
SQL> ALTER TABLE Seller ADD CONSTRAINT Un_mail UNIQUE(Email);  
Table altered.
```

Shows the unique constraint added to the Seller email column.

To make Email NOT NULL

```
SQL> ALTER TABLE Seller MODIFY Email NOT NULL;  
Table altered.
```

Shows the NOT NULL constraint applied to the Seller email column.

Add Constraint to Check the Stock Quantity is Positive

```
SQL> ALTER TABLE Inventory ADD CONSTRAINT CK_Quan CHECK(Stock_Quantity>=0);  
Table altered.
```

Shows the CHECK constraint ensuring Stock_Quantity is greater than zero.

Insert Values to Seller Table

```
SQL> INSERT INTO Seller VALUES('&Seller_ID','&Seller_Name','&email','&phone','&Address');
Enter value for seller_id: S01
Enter value for seller_name: Arun
Enter value for email: arun@gmail.com
Enter value for phone: 1234567891
Enter value for address: Chennai
old 1: INSERT INTO Seller VALUES('&Seller_ID','&Seller_Name','&email','&phone','&Address')
new 1: INSERT INTO Seller VALUES('S01','Arun','arun@gmail.com',1234567891,'Chennai')

1 row created.

SQL> /
Enter value for seller_id: S02
Enter value for seller_name: Riya
Enter value for email: riya@gmail.com
Enter value for phone: 1234567892
Enter value for address: Madurai
old 1: INSERT INTO Seller VALUES('&Seller_ID','&Seller_Name','&email','&phone','&Address')
new 1: INSERT INTO Seller VALUES('S02','Riya','riya@gmail.com',1234567892,'Madurai')

1 row created.

SQL> /
Enter value for seller_id: S03
Enter value for seller_name: Karthik
Enter value for email: karthik@gmail.com
Enter value for phone: 9876543212
Enter value for address: Coimbatore
old 1: INSERT INTO Seller VALUES('&Seller_ID','&Seller_Name','&email','&phone','&Address')
new 1: INSERT INTO Seller VALUES('S03','Karthik','karthik@gmail.com',9876543212,'Coimbatore')

1 row created.

SQL> /
Enter value for seller_id: S04
Enter value for seller_name: Divya
Enter value for email: divya@gmail.com
Enter value for phone: 9876543213
Enter value for address: Salem
```

Shows seller records being inserted into the Seller table.

```
SQL> SELECT * FROM Seller;
```

SELLER_	SELLER_NAME	EMAIL	PHONE	ADDRESS
S01	Arun	arun@gmail.com	1234567891	Chennai
S02	Riya	riya@gmail.com	1234567892	Madurai
S03	Karthik	karthik@gmail.com	9876543212	Coimbatore
S04	Divya	divya@gmail.com	9876543213	Salem
S05	Rahul	rahul@gmail.com	9876543214	Trichy

Shows the Seller table after inserting and displaying the seller records.

Report 1: Seller-wise Product Report

Display:

- Seller name
- Number of products supplied
- Available stock

```
SQL> SELECT s.Seller_Name, COUNT(i.Product_ID) AS Number_of_Products, SUM(i.Stock_Quantity) AS Available_Stock FROM Seller s JOIN Inventory i ON s.Seller_ID = i.Seller_ID WHERE i.Stock_Status = 'Available' GROUP BY s.Seller_Name;
```

SELLER_NAME	NUMBER_OF_PRODUCTS	AVAILABLE_STOCK
Arun	1	20
Divya	1	75
Riya	1	20

Shows the Seller-wise Product Report with seller name, product count, and available stock.

Use Joins to get details from Seller and Inventory Table.

Insert Values to Inventory Table

```
SQL> INSERT INTO Inventory VALUES('&Inventory_ID','&Product_ID','&Seller_ID',&Stock,'&Stock_status',TO_DATE('&Last_update','YYYY-MM-DD'));
Enter value for inventory_id: IN001
Enter value for product_id: P100
Enter value for seller_id: S01
Enter value for stock: 20
Enter value for stock_status: Available
Enter value for last_update: 2026-09-01
old 1: INSERT INTO Inventory VALUES('&Inventory_ID','&Product_ID','&Seller_ID',&Stock,'&Stock_status',TO_DATE('&Last_update','YYYY-MM-DD'))
new 1: INSERT INTO Inventory VALUES('IN001','P100','S01',20,'Available',TO_DATE('2026-09-01','YYYY-MM-DD'))

1 row created.

SQL> /
Enter value for inventory_id: IN002
Enter value for product_id: P200
Enter value for seller_id: S02
Enter value for stock: 20
Enter value for stock_status: Available
Enter value for last_update: 2008-05-05
old 1: INSERT INTO Inventory VALUES('&Inventory_ID','&Product_ID','&Seller_ID',&Stock,'&Stock_status',TO_DATE('&Last_update','YYYY-MM-DD'))
new 1: INSERT INTO Inventory VALUES('IN002','P200','S02',20,'Available',TO_DATE('2008-05-05','YYYY-MM-DD'))

1 row created.

SQL> /
Enter value for inventory_id: IN003
Enter value for product_id: P300
Enter value for seller_id: S03
Enter value for stock: 10
Enter value for stock_status: Low Stock
Enter value for last_update: 2025-09-30
old 1: INSERT INTO Inventory VALUES('&Inventory_ID','&Product_ID','&Seller_ID',&Stock,'&Stock_status',TO_DATE('&Last_update','YYYY-MM-DD'))
new 1: INSERT INTO Inventory VALUES('IN003','P300','S03',10,'Low Stock',TO_DATE('2025-09-30','YYYY-MM-DD'))

1 row created.
```

Shows inventory records being inserted into the Inventory table.

```
SQL> SELECT * FROM Inventory;

INVENTO  PRODUCT  SELLER_  STOCK_QUANTITY  STOCK_STATU  LAST_UPDA
-----  -
IN001    P100     S01      20 Available      01-SEP-26
IN002    P200     S02      20 Available      05-MAY-08
IN003    P300     S03      10 Low Stock      30-SEP-25
IN004    P400     S04      75 Available      31-JUL-26
```

Shows the Inventory table records after insertion.

Report 2: Stock Availability Report

Display:

- Product name
- Stock quantity
- Stock status

```
SQL> SELECT p.Product_Name, i.Stock_Quantity, i.Stock_Status FROM Product p JOIN Inventory i ON p.Product_ID = i.Product_ID;

PRODUCT_NAME
-----
STOCK_QUANTITY STOCK_STATU
-----
Bluetooth Speaker
      20 Available

Cotton Shirt
      20 Available

English Grammar Book
      10 Low Stock

Cricket Bat
      75 Available
```

Shows the Stock Availability Report with product name, stock quantity, and stock status.

Report 3: Inventory Analysis

Find:

- Total products available

```
SQL> SELECT COUNT(*) AS Total_Available_Products FROM Inventory WHERE Stock_Status = 'Available';

TOTAL_AVAILABLE_PRODUCTS
-----
3
```

Shows the total number of products currently available.

- Products out of stock

```
SQL> SELECT COUNT(*) AS OutOfStock_Products FROM Inventory WHERE Stock_Status = 'Out of Stock';

OUTOFSTOCK_PRODUCTS
-----
0
```

Shows the number of products that are out of stock.

- Highest stocked products

```
SQL> SELECT p.Product_Name, i.Stock_Quantity FROM Product p JOIN Inventory i ON p.Product_ID = i.Product_ID WHERE i.Stock_Quantity = (SELECT MAX(Stock_Quantity) FROM Inventory);

PRODUCT_NAME
-----
STOCK_QUANTITY
-----
Cricket Bat 75
```

Shows the product(s) with the highest stock quantity.

- Average inventory quantity

```
SQL> SELECT AVG(Stock_Quantity) AS Average_Inventory FROM Inventory;

AVERAGE_INVENTORY
-----
31.25
```

Shows the average inventory quantity calculated using AVG(Stock_Quantity).